

# MobileViews: A Million-scale and Diverse Mobile GUI Dataset

Longxi Gao<sup>1\*</sup>, Li Zhang<sup>1\*</sup>, Shihe Wang<sup>1</sup>, Pengzhi Gao<sup>3</sup>,  
Wei Liu<sup>3</sup>, Jian Luan<sup>3</sup>, Shangguang Wang<sup>1</sup>, Yuanchun Li<sup>2</sup>, Mengwei Xu<sup>1</sup>

<sup>1</sup>Beijing University of Posts and Telecommunications

<sup>2</sup>Institute for AI Industry Research (AIR), Tsinghua University

<sup>3</sup>Unaffiliated

glx@bupt.edu.cn

## Abstract

Visual language models (VLMs) empower mobile GUI agents to interpret complex mobile screens and respond to user requests. Training such capable agents requires large-scale, high-quality mobile GUI data. However, existing mobile GUI datasets are limited in scale, data comprehensiveness, and fidelity. To overcome this, we utilize two mobile SoC clusters to provide over 200 native, high-fidelity mobile environments, along with a VLM-enhanced automatic application traversal framework for highly parallel, automated dataset collection with minimal human intervention. With this system, we propose MobileViews, a million-scale mobile GUI dataset comprising over 1.2 million unique screenshot-view hierarchy pairs from more than 30K modern Android applications. We assess the effectiveness of MobileViews by training four VLMs using the reinforcement learning-based GUI grounding task and evaluating them on two representative GUI grounding benchmarks. Results show that MobileViews significantly enhances grounding accuracy by up to 6.1%. Further analysis of data scale and quality underscores the critical role of large, high-quality datasets as reliable sources for training mobile GUI agents. The MobileViews dataset is publicly available at <https://huggingface.co/datasets/mlmTeam/MobileViews>.

## 1. Introduction

Visual language models (VLMs) empower mobile GUI agents to understand screens and respond to diverse user requests, benefiting individuals with disabilities and hands-free scenarios such as driving [15, 21, 38]. During pre-training, general-purpose VLMs gain basic GUI understanding and reasoning abilities by incorporating a small portion of GUI data within billion-scale natural image-text pairs [5, 7, 36].

\*Equal contribution.

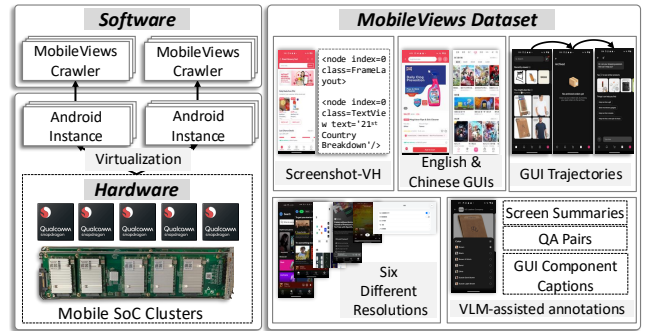


Figure 1. Overview of MobileViews and its collection infrastructure. We employ the MobileViews Crawler to automatically interact with native Android instances, which are powered by Android virtualization-enhanced Mobile SoC Clusters.

However, natural images depicting real-world scenes still dominate the training corpus, while mobile screens with dense layouts, structured components, and text-rich content remain severely underrepresented. This imbalance limits the models’ ability to handle complex GUI tasks. Therefore, large-scale and comprehensive GUI data may empower general-purpose VLMs to handle GUI-specific tasks.

**Existing datasets and limitations.** As shown in Table 1, existing open-source mobile GUI datasets face a critical trade-off between scale and comprehensiveness. Rico [9], proposed in 2017, includes only 63K unique screens at a single screen resolution. While AITW [27] contains 2.28M unique screens, it suffers from two key limitations: (1) limited app diversity results in high content similarity (as we will shown in §4), and (2) the lack of view hierarchies (VHs) hinders fine-grained and accurate screen interpretation in downstream training tasks. GUIOdyssey [23] and Android-Control [18] provide rich textual annotations but rely on manual construction, leading to prohibitive costs and limited scalability. Additionally, most datasets contain only English GUIs and single-resolution screens, lacking the linguistic and visual diversity needed for generalized GUI agents.

Table 1. Comparison of representative open-source mobile GUI datasets. For large datasets (i.e., those with more than 50,000 screens), we count the number of unique screens using ImageHash [6] to eliminate duplicates.

| Mobile GUI Dataset  | Scale         |                  | Data Comprehensiveness |                     |                  |                      |                     | Data Collection |                     |
|---------------------|---------------|------------------|------------------------|---------------------|------------------|----------------------|---------------------|-----------------|---------------------|
|                     | Apps          | Unique Screens   | App Metadata           | Screenshot-VH Pairs | GUI Trajectories | Multiple Resolutions | Languages (EN / ZH) | Automation      | Hardware            |
| Rico [9]            | 9,772         | 63,370           | ✓                      | ✓                   | ✓                | ✗                    | EN                  | ✗               | Physical Devices    |
| PixelHelp [20]      | 4             | 187              | ✗                      | ✓                   | ✓                | ✗                    | EN                  | ✗               | Emulators           |
| Screen2words [31]   | 6,269         | 22,417           | ✓                      | ✓                   | ✗                | ✗                    | EN                  | ✗               | N/A                 |
| ScreenQA [4]        | N/A           | 35,352           | ✓                      | ✓                   | ✗                | ✗                    | EN                  | ✗               | N/A                 |
| META-GUI [29]       | 11            | 24,825           | ✗                      | ✓                   | ✓                | ✗                    | EN                  | ✗               | Physical Devices    |
| DroidTask [32]      | 13            | 362              | ✗                      | ✓                   | ✓                | ✗                    | EN                  | ✗               | Emulators           |
| AITW [27]           | 357           | 2,282,533        | ✗                      | ✗                   | ✓                | ✓                    | EN                  | ✗               | Emulators           |
| LlamaTouch [40]     | 57            | 3,281            | ✗                      | ✓                   | ✓                | ✗                    | EN                  | ✗               | Emulators           |
| GUIOdyssey [23]     | 212           | 77,535           | ✗                      | ✗                   | ✓                | ✓                    | EN                  | ✗               | Emulators           |
| AndroidControl [18] | 833           | 81,459           | ✗                      | ✓                   | ✓                | ✗                    | EN                  | ✗               | Physical Devices    |
| GUI-Net-1M [37]     | 280           | 720,374          | ✗                      | ✗                   | ✓                | ✓                    | EN & ZH             | ✗               | N/A                 |
| <b>MobileViews</b>  | <b>30,037</b> | <b>1,213,866</b> | ✓                      | ✓                   | ✓                | ✓                    | <b>EN &amp; ZH</b>  | ✓               | <b>SoC clusters</b> |

These limitations call for a large-scale and comprehensive mobile GUI dataset built upon a scalable collection pipeline.

**Challenges and key techniques.** Collecting large-scale mobile GUI datasets faces fundamental challenges due to inefficiencies at both the software and hardware levels.

- At the software level, efficient and scalable traversal across diverse mobile apps is essential, yet current solutions do not fully address this requirement. (1) Static rule-based GUI navigation tools [1, 19] often yield low-quality screen data, as they struggle to adapt to complex GUI contexts and diverse navigation patterns. For instance, they frequently get stuck on login pages or intrusive pop-ups. (2) Most existing large mobile GUI datasets [9, 18, 23, 27] are manually collected to ensure high quality, as shown in Table 1. However, such manual collection process is not scalable due to the substantial human labor required.

**Design#1: VLM-enhanced automatic app crawler.** To ensure both efficiency and data quality, we design MobileViews Crawler, a VLM-enhanced automatic traversal framework that follows predefined interaction rules and invokes VLM to process a list of pre-defined triggers. Specifically, the crawler captures the VH metadata, simplifies it, identifies and traverses interactable GUI components sequentially. When predefined triggers occur, such as prolonged idleness, login pages, or pop-ups, the VLM is activated to assist further traversal. This hybrid strategy maximizes GUI coverage and minimizes manual intervention, which is required only when both predefined rules and the VLM fail to proceed.

- At the hardware level, large-scale app traversal requires running numerous Android instances in parallel with high environment fidelity. Most mobile GUI datasets rely on emulators [27, 29, 30, 40] for cost-effective scaling. However, emulators typically run on x86 machines, causing compatibility issues with ARM-based apps and limiting app diversity. For instance, AITW [27] collects about 5M screens from only 357 apps on Android emulators, but fewer than 2.3M are left after image de-duplication. In contrast, Rico [9] uses physical device farms to run apps natively, enabling collection from over 9K apps. Although physical devices

eliminate compatibility constraints, they face scalability challenges due to high operational costs and space constraints.

**Design#2: Scalable, high-fidelity app traversal on SoC clusters.** To improve hardware-level traversal efficiency, we deploy two SoC clusters [39] with 120 Snapdragon processors [3], which provide native Android environments and high app compatibility. On top of this, we leverage Android OS virtualization to launch over 200 instances across the two SoC clusters. Together with fine-grained task scheduling, this setup achieves high parallelism and significantly reduces data collection time, which gathers over 1.2M unique screens in approximately 151,600 device-hours.

**The new dataset: MobileViews.** Based on the proposed system, we construct MobileViews, an open-source dataset with over 1.2M unique screenshot-VH pairs from more than 30K Android apps. Compared to AndroidControl [18], MobileViews includes 36× more apps and 15× more screenshot-VH pairs, with a scalable pipeline for flexible expansion. Beyond scale, MobileViews covers six screen resolutions, both English and Chinese GUIs, and complete GUI trajectories. We further enhance MobileViews with 13K screen summaries, 26K QA pairs, and 26K GUI component captions. Detailed dataset statistics are presented in §4.

**Experiments.** We evaluate MobileViews through GUI grounding experiments on four VLMs: three general-purpose models (Qwen2.5-VL-7B [5], MIMO-VL-7B-SFT [36], MIMO-VL-7B-RL [36]) and one GUI-specific model (UI-Tars-1.5 [26]). All models are reinforcement fine-tuned on the annotated subset and evaluated on two GUI grounding benchmarks, ScreenSpot-v2 [34] and ScreenSpot-Pro [17]. Results demonstrate that training with MobileViews consistently enhances GUI grounding performance across all models. After fine-tuning, UI-Tars-1.5 achieves 91.2% on ScreenSpot-v2 and 47.7% on ScreenSpot-Pro, with improvements of 2.2% and 6.1%, respectively. With an additional training data filtering strategy applied, MIMO-VL-7B-SFT improves from 87.6% to 91.0% on ScreenSpot-v2, while MIMO-VL-7B-RL improves from 40.2% to 43.5% on ScreenSpot-Pro. These findings confirm that MobileViews

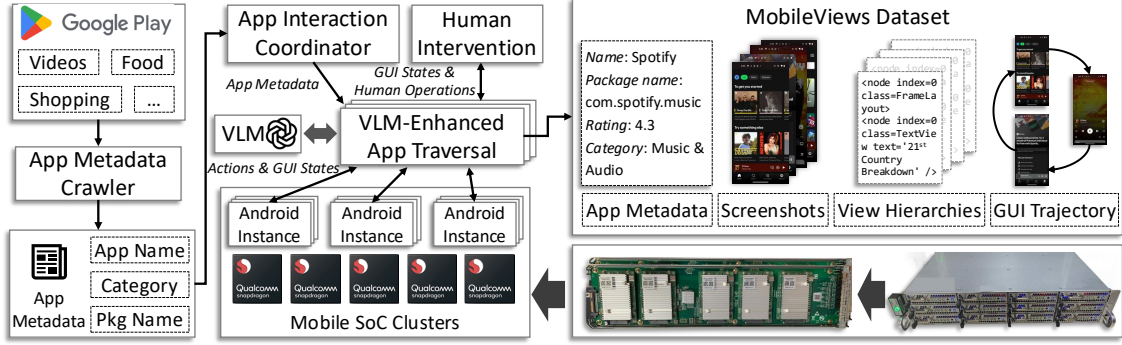


Figure 2. Left: Workflow of the MobileViews dataset construction. Right: Demonstration of the MobileViews dataset, and a physical SoC Cluster to empower high parallelism app traversal.

serves as a large-scale, high-quality dataset for improving both general-purpose and GUI-specific VLMs.

**Contributions** of this study are as follows:

- A framework with a joint software-hardware design for high-fidelity, automated mobile GUI data collection.
- MobileViews, a million-scale mobile GUI dataset comprising over 1.2M unique screens from more than 30K Android apps, will be fully open-sourced.
- Experiments on four VLMs demonstrate that training with MobileViews improves GUI grounding performance across models by up to 6.1%.

## 2. Related Work

In this section, we compare existing open-source mobile GUI datasets from three perspectives: (1) data scale, (2) data comprehensiveness, and (3) data collection methods. Table 1 summarizes these key characteristics.

**Data scale.** Rico [9] is a widely-used dataset released in 2017, containing about 63K unique screens collected from over 9K Android apps. Several studies have augmented Rico with additional annotations, such as Screen2Words [31] and ScreenQA [16], but remain limited in scale. Beyond static screen understanding, some datasets [18, 20, 23, 27, 33, 40] focus on training and benchmarking GUI grounding or navigation models. However, most of these datasets are collected from a limited number of apps and contain relatively few screens, yet recent GUI-specific VLMs demonstrate that large-scale, high-quality data is critical for effective GUI understanding. For instance, UI-Venus [12] selects only 107K high-quality samples from 627K raw GUI grounding data to improve GUI grounding performance, showing that effective quality control requires sufficiently large data pools.

**Data comprehensiveness.** From an informational perspective, both pixel-level screenshots and textual VHs are essential for mobile GUI understanding. While screenshots capture the exact visual state of a screen and provide a quick reference for layout and image-based components, VHs record the screen’s structural metadata, including component

hierarchy, bounding boxes, and attributes that are not directly visible. Together, these two modalities offer a comprehensive understanding of both the visual and structural aspects of mobile screens. Although AITW [27] reaches the largest scale to date, about 2.3M screens from 357 apps, it lacks VH metadata, preventing flexible sample construction by combining both visual and structural information. Similarly, GUI-Net-1M [18], which is collected from online videos (e.g., YouTube, Bilibili) instead of direct app traversal, also lacks VHs. Moreover, most datasets do not account for the diversity of screen resolutions or languages, restricting models’ cross-device and cross-language generalization. Overall, existing datasets are far from comprehensive in capturing both visual and structural aspects of mobile GUIs.

**Data collection methods.** The data collection platforms and strategies are both crucial for ensuring the quality and scalability of mobile GUI datasets. Datasets such as Rico [9] rely on physical devices, which provide high-fidelity environments but face scalability challenges due to cost and space constraints. AITW [27] and GUIOdyssey [23] use emulators to collect data, which offers cost-effective scaling but suffers from compatibility issues since most mobile apps are designed for ARM processors, whereas emulators typically run on x86 machines. This incompatibility restricts app diversity and limits the coverage of modern apps. Beyond environment constraints, some datasets rely on manual collection, which is prohibitively time-consuming and costly. For instance, AndroidControl [18] takes one year to collect 14,548 episodes over 833 Android apps. These limitations motivate the construction of MobileViews, which combines VLM-enhanced automatic app traversal with high-fidelity SoC clusters to enable scalable and reliable data collection across diverse modern mobile applications.

## 3. MobileViews Dataset Construction

Given the limitations of previous datasets in terms of scale and comprehensiveness, we construct a new mobile GUI dataset, MobileViews. It contains over 1.2M unique

Table 2. Comparison of three different app traversal methods. #Apps is the number of apps successfully traversed without stuck.

| Metrics            | Fully Rule-based | W/ VLM | W/ Human Intervention |
|--------------------|------------------|--------|-----------------------|
| #Apps (Total = 86) | 16               | 38     | 86                    |
| Success Rate       | 18.6%            | 44.2%  | 100%                  |

screenshot-VH pairs collected from more than 30K modern mobile apps, covering six different resolutions, both English and Chinese GUIs, and complete GUI trajectories. Building such a large-scale dataset poses a key challenge: *how to efficiently collect large-scale, comprehensive mobile screens with minimal human intervention*. To address this, we propose MobileViews Crawler, a scalable app traversal framework for automated data collection. As illustrated in Figure 2, it includes a *VLM-enhanced automatic app traversal tool* that processes individual apps on each Android instance by combining fixed interaction rules, VLM, and minimal human intervention to achieve efficient app traversal. This workflow is further scaled across two SoC clusters to enable high-parallel app exploration.

### 3.1. App Traversal Workflow

**App metadata collection.** The data collection process begins with gathering app metadata, such as app package names. MobileViews covers apps from 33 categories on the Google Play Store, including many trending ones, providing a more accurate reflection of recent mobile interface designs.

**App interaction coordination.** The collected metadata is sent to an *App Interaction Coordinator*, which serves as the overall traversal task manager. The coordinator manages Android instances provided by the SoC clusters and initializes each traversal process. Each process starts with app installation and launching. Specifically, the coordinator sends an app package name (e.g., *com.spotify.music*) to the traversal tool on an available Android instance. The traversal tool then searches for this app on the Google Play Store using GUI automation scripts, installs it, and launches it via the Android Debug Bridge [2]. Once launched, the traversal tool begins interacting with the app to collect screenshots and VHs. If the step limit is reached or an unexpected interruption occurs, the traversal tool stops. The coordinator will record the current system and app states for later analysis.

**VLM-enhanced automatic app traversal.** By default, the traversal tool follows predefined rules to interact sequentially with all interactable GUI components on each screen. When the traversal gets stuck, such as on login pages, the VLM steps in to interpret complex layouts and guide subsequent interactions to access deeper interfaces.

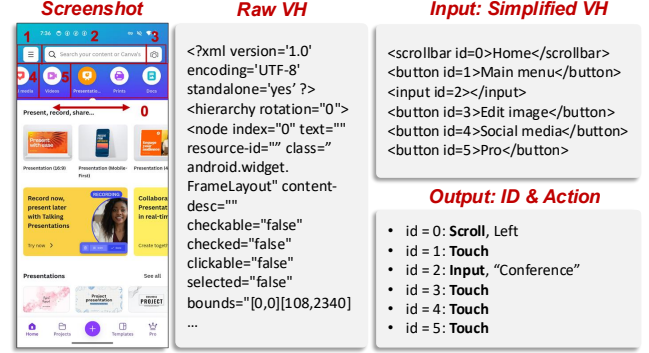


Figure 3. Example of a single screen interaction. The traversal tool simplifies the raw VH and identifies the interactable GUI components along with their corresponding actions.

### 3.2. App Interaction with MobileViews Crawler

Traditional app traversal methods rely heavily on human effort to ensure data quality. However, such manual approaches are not scalable when building datasets involving a large number of apps and screens. Our proposed MobileViews Crawler, a VLM-enhanced automatic app traversal framework, minimizes human intervention while maintaining both efficiency and scalability.

**Rule-based app interaction.** We employ a rule-based app traversal tool, Droidbot [19], to automate the traversal process. As illustrated in Figure 3, Droidbot captures the VH of the current screen and identifies interactable GUI components based on specific attributes such as *clickable*, *editable*, and *focusable*. It then executes corresponding actions (e.g., clicks, scrolls, or text inputs) on these GUI components in sequence. When Droidbot navigates to a new screen, it repeats this procedure until reaching the maximum number of steps, which is set to 1,000 actions per app in our setup.

**VLM integration.** While fixed interaction rules can reach many common GUIs effectively, they often fail to handle complex ones, such as accurately filling in specific input fields or closing intrusive pop-ups. As shown in Table 2, the rule-based approach successfully traverses only 18.6% (16 out of 86 trending apps on the Google Play Store) apps without getting stuck. This low success rate arises from multiple challenges: many apps require registration or login to access deeper interfaces, intrusive pop-ups block navigation, and unintended links to external apps disrupt the traversal flow. To address this, we integrate a VLM to handle complex GUI states under two primary conditions: (1) the traversal tool encounters GUI states matching predefined triggers (e.g., keywords like “sign up” or “login” are detected in the VH), and (2) the current screen remains idle beyond a defined timeslot. In these cases, the VLM takes over the traversal process until it reports completion, either successful or failed. Utilizing GPT-4o [25], the traversal success rate increases to 44.2% (38 out of 86 apps). This VLM-enhanced framework



can also be extended to other scenarios by defining suitable triggers to handle various complex GUI states.

**Human intervention.** Although the VLM integration can handle complex GUI states effectively, certain challenging cases can still cause the traversal process to stall. This typically occurs when the VLM reaches its maximum retry limit or returns a “task failed” response. To address such situations, we adopt a *lazy processing* strategy in which human operators intervene only when the traversal tool encounters an unprocessable GUI state. In these cases, the tool stops exploration and the *App Interaction Coordinator* records the current state. Human operators later review these apps in batches, manually resolve the blocked states, and resume automatic traversal. This strategy significantly reduces manual effort while maintaining traversal coverage and efficiency.

### 3.3. SoC Cluster-Powered Scalable App Interaction

By combining rule-based interaction, VLM, and human intervention, the traversal tool can efficiently navigate each app and collect diverse GUIs. The next challenge lies in scaling this process to handle large number of apps in native mobile environments. Existing approaches for mobile GUI dataset construction, such as running apps on physical device farms or Android emulators, both have notable limitations. While physical device farms are costly to scale, Android emulators running on x86-based machines often face compatibility issues with apps built for ARM-based processors. To ensure both scalability and compatibility, we deploy two SoC clusters [39] equipped with 120 Snapdragon [3] processors, which provide native ARM-based execution environments for large-scale mobile app interaction. Figure 2 shows the physical setup of an SoC cluster and its network-interconnected mobile processors inside the server. To maximize resource utilization, we further implement SoC-level virtualization to run multiple Android instances on each processor. As a result, more than 200 Android instances run in parallel across the two SoC clusters, enabling large-scale app interaction in a more efficient and cost-effective manner than traditional device farms.

**Software-hardware coordination.** To achieve better parallelism, the *App Interaction Coordinator* monitors the status of each Android instance at regular intervals. When an instance becomes available, the coordinator connects to it and initiates the app traversal process. The traversal tool then starts its execution. Once the traversal is completed or the process exits unexpectedly, the instance is marked as available again. This fine-grained scheduling strategy enables efficient utilization of hardware resources, allowing us to complete 151,600 device-hours of app traversal within two months.

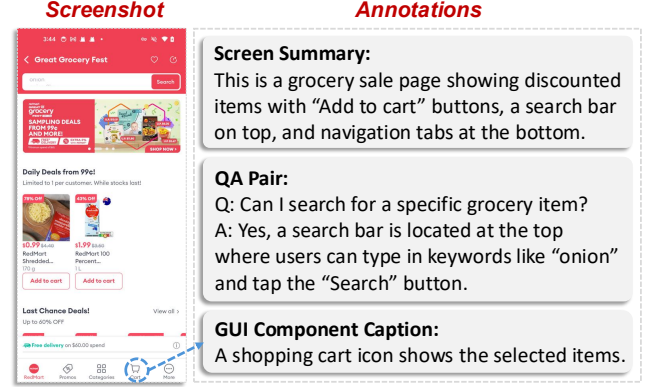


Figure 4. Illustration of VLM-assisted annotations, including a screen summary, a QA pair, and a GUI component caption.

## 4. MobileViews Dataset

As shown in Table 1, MobileViews contains over 1.2M unique mobile screens collected from more than 30K Android apps, demonstrating clear advantages over existing datasets in scale, comprehensiveness, and collection scalability. In this section, we detail MobileViews’s composition and analyze its screen diversity.

**MobileViews composition and features.** MobileViews is a large-scale and diverse resource for training mobile GUI agents with over 1.2M mobile screenshot-VH pairs. It also provides GUIs across multiple resolutions, languages, and trajectories. Specifically, screenshots capture the visual state of each mobile interface, offering a direct view of layouts and image-based components. VHs contain detailed structural metadata describing GUI component hierarchies and attributes, including bounding boxes, component types, and interaction properties, serving as a valuable foundation for constructing diverse training samples. Multiple resolutions enhance model generalization across diverse device configurations. For English GUIs, MobileViews includes five resolution settings:  $720 \times 1280$ ,  $720 \times 1520$ ,  $1080 \times 2340$ ,  $1440 \times 2560$ , and  $1920 \times 1080$ , each containing more than 60K samples, plus an additional  $1080 \times 1920$  setting covering the remaining English screens. Chinese GUIs comprise 50K unique screens, all at  $1080 \times 1920$  resolution, strengthening bilingual support. GUI trajectories are naturally captured during the automated traversal process, with each trajectory recording the sequence of screens and interactions to provide valuable contextual information for studying GUI navigation patterns. By integrating all these features at scale, MobileViews offers high data diversity and broad coverage, making it suitable for training capable mobile GUI agents.

**Enhance MobileViews with VLM-assisted annotations.** To support various GUI tasks, we augment MobileViews with VLM-assisted annotations for both mobile GUI understanding and grounding. First, we select 20K unique screens from MobileViews as annotation candidates. We then ex-

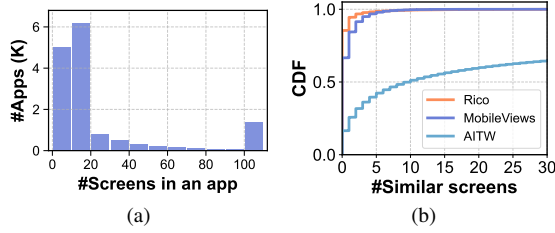


Figure 5. Screen diversity analysis. (a) Distribution of the number of screens collected per app in MobileViews. (b) Cumulative distribution function (CDF) of similar screens in Rico, AITW, and MobileViews, where the X-axis denotes the number of similar screens corresponding to each screen in a dataset.

tract metadata from each screen’s VH, including component interactability, bounding boxes, and text labels, which serve as the basis for multi-stage filtering. To ensure data diversity and annotation quality, this filtering proceeds in two steps:

- *Screen-level filtering.* We exclude screens with fewer than three interactable GUI components to ensure sufficient content richness for meaningful annotations.
- *Component-level filtering.* We retain only clickable GUI components with non-empty text descriptions and remove components with duplicate labels on the same screen to reduce annotation ambiguity. We further use OCR to verify that component text is clearly visible and readable, filtering out components with obscured text.

After filtering, we obtain 13K representative screens with clear and diverse GUI components. We then employ GPT-4o [25] to generate annotations for each screen: one screen summary describing the overall interface, two QA pairs about screen functionality, and two GUI component captions describing selected GUI components. This produces an annotated subset of 13K screens with corresponding summaries, 26K QA pairs, and 26K GUI component captions, providing reliable supervision for downstream GUI understanding and grounding tasks. Figure 4 illustrates examples of the VLM-assisted annotations.

**Screen diversity analysis.** We examine MobileViews’s screen diversity from two perspectives: (1) the distribution of screens collected per app, and (2) screen similarity across large-scale mobile GUI datasets. The second perspective is motivated by the fact that, before image de-duplication, AITW contains over 5M screens from only 357 apps, suggesting a considerable amount of visually similar content. To measure similarity, we use *ImageHash* [6] to compute a perceptual hash for each screen, and treat two screens as similar if their hashes are identical. Below, we examine these two aspects through quantitative analysis.

- *How many screens are collected per app in MobileViews?* Figure 5a shows the distribution of screens collected per app. Most apps contribute 10–20 unique screens, reflecting the natural variation in app complexity: simple apps (e.g., calculators) expose fewer screens, whereas complicated apps

(e.g., social media) expose more. Notably, 6.8% of apps yield over 100 screens, indicating that our VLM-enhanced traversal is able to explore apps extensively. Overall, this distribution demonstrate that MobileViews achieves broad app coverage while ensuring GUI coverage within apps.

- *How does screen uniqueness compare across datasets?*

Figure 5b compares screen similarity across Rico [9], AITW [27], and MobileViews. For each screen, we count how many visually similar screens appear in the same dataset and use the cumulative distribution function (CDF) to summarize these counts. Both MobileViews and Rico exhibit high screen uniqueness: over 95% of screens have fewer than 5 similar counterparts, and the curves saturate rapidly due to their broad app coverage. In contrast, AITW shows severe redundancy. The CDF rises slowly and plateaus around 0.7, meaning 30% of AITW’s screens have more than 30 similar duplicates. More specifically, over 1.2M screens (24% of the dataset) have over 100 similar counterparts, and the most duplicated screen appears 67K times, accounting for 1.3% of the dataset. These comparisons show that MobileViews balances scale and diversity, providing a dataset suitable for training mobile GUI agents that require broad perceptual coverage and stronger generalization.

## 5. Experiments

As described in §4, we select 13K representative screens from MobileViews and construct 26K GUI grounding samples through multi-stage filtering and VLM-assisted annotation. To assess the data quality of MobileViews, we train four VLMs on these samples and evaluate their performance on established GUI grounding benchmarks. In this section, we first describe the training algorithm and experimental setup. We then (1) compare our trained VLMs with state-of-the-art models, and (2) investigate how training data filtering strategy and data scale affect VLM performance.

### 5.1. Training Algorithm

In realistic GUI interactions, a click is considered successful when the predicted point falls within the target GUI component. However, supervised fine-tuning (SFT) enforces an exact token-level match between the model output and the ground truth, which diverges from this practical interaction criterion. To better align training objectives with real-world execution, we adopt the Group Relative Policy Optimization (GRPO) algorithm, optimized in DeepSeek-R1 [13]. GRPO removes the need for an additional reward model by using structured, verifiable rewards, which substantially reduces training overhead while allowing the reward signal to be directly tied to whether the predicted point hits the correct GUI area. Several recent studies [10, 12, 22, 24, 35] further validate the effectiveness of GRPO by introducing customized reward functions tailored to GUI tasks. In this study, we employ GRPO with a task-specific reward function that reflects

the hit accuracy of predicted points in GUI grounding.

**Training objectives.** Following DeepSeek-R1 [13], our training objective aims to maximize the expected clipped advantage reward while penalizing divergence from a reference policy. Specifically, given a GUI grounding question  $q$ , we sample  $G$  responses  $\{o_i\}_{i=1}^G$  from the old policy  $\pi_{\theta_{old}}$ . For each sample, we compute the probability ratio  $\rho_i = \frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}$ , and the advantage  $A_i = \frac{r_i - \text{mean}(\{r_1, \dots, r_G\})}{\text{std}(\{r_1, \dots, r_G\})}$ , where  $r_i$  is the reward assigned to the  $i$ -th response based on our designed reward function.  $\epsilon$  is the clipping parameter to stabilize training,  $\beta$  is the KL penalty coefficient, and  $\pi_{\text{ref}}$  is the reference policy used for KL divergence calculation. The overall GRPO objective is defined as:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E} \left[ q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O | q) \right] \frac{1}{G} \sum_{i=1}^G \left[ \min(\rho_i A_i, \text{clip}(\rho_i, 1 - \epsilon, 1 + \epsilon) A_i) - \beta \mathbb{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right] \quad (1)$$

**Reward design for GUI grounding.** In GUI grounding, a click is deemed successful if the predicted point lies within the target GUI component’s bounding box. Accordingly, we design a straightforward reward function that assigns a reward of 1 if the predicted point falls within the ground truth bounding box, and 0 otherwise:

$$R = \begin{cases} 1, & \text{if point in bounding box} \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

This binary reward effectively guides the model to learn accurate grounding strategies aligned with practical interaction requirements. Different from previous studies [10, 12, 13, 22, 24, 35] that include a format reward to enforce output consistency and facilitate result parsing, we observe that, when using the official GUI grounding prompts provided for each model, the models are able to produce the required output format consistently from the start of training. Therefore, we do not include the external format correctness reward and instead require the model to predict the center point directly, enabling it to focus exclusively on the grounding objective.

## 5.2. Experimental Setup

**Training data selection.** To validate the effectiveness of MobileViews in enhancing GUI grounding capabilities and to investigate how data quality and scale affect model performance, we design two comparative settings:

- *Random scaling.* To examine the impact of data scale, we randomly sample subsets of different sizes from the constructed 26K dataset for training and analyze performance trends as the scale increases.

- *Model-specific filtering.* To assess the effect of data quality, we perform model-specific data filtering before GRPO training to select samples better aligned with each model’s learning capacity. Specifically, for each GUI grounding sample, we use the base model to generate  $G = 8$  responses and score them with the reward function defined in Eq. 2, producing a total score between 0 and 8. Samples with scores of 0 or 8 are discarded, as they represent either overly difficult or trivial cases that provide limited learning value, while samples with intermediate scores (1–7) are retained for subsequent training.

**Training setup.** To evaluate the impact of MobileViews on both general-purpose and GUI-specific VLMs, we select four models: Qwen2.5-VL-7B [5], MiMo-VL-7B-SFT [36], MiMo-VL-7B-RL [36], and UI-Tars-1.5 [26]. All models participate in the random data scaling experiments. For the model-specific data filtering experiments, we apply the filtering strategy individually to each model and obtain filtered subsets containing 1K, 1.3K, and 254 samples for MiMo-VL-7B-SFT, MiMo-VL-7B-RL, and UI-Tars-1.5, respectively. For Qwen2.5-VL-7B, we observe that all samples receive either 0 or 8 reward scores when using a high sampling temperature; therefore, this model is only included in the data scaling experiments. We implement the GRPO training based on the VLM-R1 [28] framework with the vision encoder frozen and the language model fully fine-tuned. We train each model for four epochs with sampling temperature 0.9, KL penalty  $\beta = 0.04$ , and eight responses per sample. All experiments are conducted on  $8 \times \text{H100 GPUs}$ .

**Benchmarks and metrics.** We evaluate all models on two established GUI grounding benchmarks: ScreenSpot-V2 [34], a refined version of ScreenSpot [8] with corrected annotations, containing 1,272 samples from mobile, desktop, and web platforms; and ScreenSpot-Pro [17], a more challenging benchmark comprising 1,581 samples focusing on high-resolution grounding. We adopt center-point accuracy as the evaluation metric, which measures the proportion of predicted points that fall within the ground-truth bounding boxes of the target GUI components.

## 5.3. Results and Analysis

We first validate the effectiveness of MobileViews in improving GUI grounding performance, then analyze how data quality and scale impact model performance.

**Comparison with state-of-the-art models.** Table 3 summarizes the results on ScreenSpot-v2 and ScreenSpot-Pro for performant VLMs and four models trained with MobileViews. In general, models trained with MobileViews show clear improvements over their original versions and perform competitively with prior strong models. On ScreenSpot-v2, MiMo-VL-7B-SFT [36] trained with MobileViews surpasses UI-Venus-Ground-7B [12] in both mobile and web text grounding, achieving an average accuracy of 91.0%. On



Table 3. Performance comparison on GUI grounding benchmarks: ScreenSpot-v2 and ScreenSpot-Pro. Training with MobileViews enhances grounding ability across both general-purpose and GUI-specific VLMs. **Bold**: best result; underlined: second best.

| Model                                             | ScreenSpot-v2 |             |             |             |             |             |                      | -Pro                 |
|---------------------------------------------------|---------------|-------------|-------------|-------------|-------------|-------------|----------------------|----------------------|
|                                                   | Mobile        |             | Desktop     |             | Web         |             | Avg.                 | Avg.                 |
|                                                   | Text          | Icon        | Text        | Icon        | Text        | Icon        |                      |                      |
| CogAgent-18B [14]                                 | -             | -           | -           | -           | -           | -           | -                    | 7.7                  |
| SeeClick-9.6B [8]                                 | 78.4          | 50.7        | 70.1        | 29.3        | 55.2        | 32.5        | 55.1                 | 1.1                  |
| UGround-7B [11]                                   | 75.1          | 84.5        | 85.1        | 61.4        | 84.6        | 71.9        | 76.3                 | 16.5                 |
| OS-Atlas-7B [34]                                  | 95.2          | 75.8        | 90.7        | 63.6        | 90.6        | 77.3        | 84.1                 | 18.9                 |
| ShowUI-2B [21]                                    | -             | -           | -           | -           | -           | -           | -                    | 7.7                  |
| UI-R1-E-3B [24]                                   | 83.0          | <b>97.1</b> | 85.0        | <b>91.7</b> | 77.9        | <b>95.4</b> | 89.5                 | 33.5                 |
| InfGUI-R1-3B [22]                                 | -             | -           | -           | -           | -           | -           | -                    | 35.7                 |
| TongUI-7B [37]                                    | 93.1          | 81.5        | 96.4        | 82.9        | 90.2        | 84.7        | 88.7                 | 24.7                 |
| UI-Venus-Ground-7B [12]                           | <u>99.0</u>   | <u>90.0</u> | <b>97.0</b> | <u>90.7</u> | <u>96.2</u> | <u>88.7</u> | <b>94.1</b>          | <b>50.8</b>          |
| <i>Ours: Four models trained with MobileViews</i> |               |             |             |             |             |             |                      |                      |
| Qwen2.5-VL-7B [5]                                 | 98.3          | 86.3        | 88.7        | 67.1        | 92.7        | 81.8        | 87.7                 | 26.4                 |
| w/ MobileViews                                    | <u>99.0</u>   | 84.4        | 89.2        | 75.7        | 94.4        | 81.8        | 89.2 <sup>+1.5</sup> | 28.4 <sup>+2.0</sup> |
| Mimo-VL-7B-SFT [36]                               | 96.6          | 84.4        | 92.8        | 80.0        | 88.9        | 76.8        | 87.6                 | 39.8                 |
| w/ MobileViews                                    | <b>99.3</b>   | 85.3        | 93.3        | 81.4        | <b>96.6</b> | 83.3        | 91.0 <sup>+3.4</sup> | 42.1 <sup>+2.3</sup> |
| Mimo-VL-7B-RL [36]                                | 98.3          | 86.3        | 90.2        | 80.7        | 92.7        | 75.4        | 88.4                 | 40.2                 |
| w/ MobileViews                                    | <b>99.3</b>   | 88.2        | 94.3        | 82.1        | 92.3        | 76.4        | 89.9 <sup>+1.5</sup> | 43.5 <sup>+3.3</sup> |
| UI-Tars-1.5 [26]                                  | 95.5          | 84.4        | 91.2        | 78.6        | 91.5        | 86.7        | 89.0                 | 41.6                 |
| w/ MobileViews                                    | 98.3          | 82.9        | 94.3        | 79.3        | 93.6        | 86.7        | 90.3 <sup>+1.3</sup> | 46.1 <sup>+4.5</sup> |

Table 4. Performance comparison on GUI grounding benchmarks: ScreenSpot-v2 and ScreenSpot-Pro. Each model is trained with the same number of samples, using either unfiltered or filtered data from MobileViews. Filtering improves data quality and leads to consistent performance gains across all models.

| Model                                   | ScreenSpot-v2 |      |         |      |      |      |                      | -Pro                 |
|-----------------------------------------|---------------|------|---------|------|------|------|----------------------|----------------------|
|                                         | Mobile        |      | Desktop |      | Web  |      | Avg.                 |                      |
|                                         | Text          | Icon | Text    | Icon | Text | Icon |                      |                      |
| Mimo-VL-7B-SFT trained with 1K samples  |               |      |         |      |      |      |                      |                      |
| Mimo-VL-7B-SFT [36]                     | 96.6          | 84.4 | 92.8    | 80.0 | 88.9 | 76.8 | 87.6                 | 39.8                 |
| w/o data filtering                      | 99.0          | 86.7 | 92.3    | 79.3 | 91.5 | 76.8 | 88.8 <sup>+1.2</sup> | 40.7 <sup>+0.9</sup> |
| w/ data filtering                       | 99.3          | 85.3 | 93.3    | 81.4 | 96.6 | 83.3 | 91.0 <sup>+3.4</sup> | 42.1 <sup>+2.3</sup> |
| Mimo-VL-7B-RL trained with 1.3K samples |               |      |         |      |      |      |                      |                      |
| Mimo-VL-7B-RL [36]                      | 98.3          | 86.3 | 90.2    | 80.7 | 92.7 | 75.4 | 88.4                 | 40.2                 |
| w/o data filtering                      | 99.3          | 86.7 | 92.3    | 80.7 | 93.6 | 76.8 | 89.5 <sup>+1.1</sup> | 41.6 <sup>+1.4</sup> |
| w/ data filtering                       | 99.3          | 88.2 | 94.3    | 82.1 | 92.3 | 76.4 | 89.9 <sup>+1.5</sup> | 43.5 <sup>+3.3</sup> |
| UI-Tars-1.5 trained with 254 samples    |               |      |         |      |      |      |                      |                      |
| UI-Tars-1.5 [26]                        | 95.5          | 84.4 | 91.2    | 78.6 | 91.5 | 86.7 | 89.0                 | 41.6                 |
| w/o data filtering                      | 96.2          | 83.9 | 92.8    | 79.3 | 91.5 | 87.2 | 89.5 <sup>+0.5</sup> | 41.7 <sup>+0.1</sup> |
| w/ data filtering                       | 97.2          | 83.9 | 93.3    | 82.1 | 92.7 | 86.7 | 90.3 <sup>+1.3</sup> | 43.0 <sup>+1.4</sup> |

ScreenSpot-Pro, which requires high-resolution grounding, all four models trained with MobileViews achieve consistent improvements ranging from 2.0% to 4.5%. Notably, UI-Tars-1.5 [26] improves from 89.0% to 90.3% on ScreenSpot-v2 and from 41.6% to 46.1% on ScreenSpot-Pro, indicating that MobileViews benefits models already optimized for GUI tasks. These results demonstrate that MobileViews consistently enhances GUI grounding across both general-purpose and GUI-specific VLMs.

**Effects of model-specific training data filtering.** To isolate the impact of filtering itself, we train each model with an equal number of unfiltered samples and directly compare their performance with the filtered counterparts.

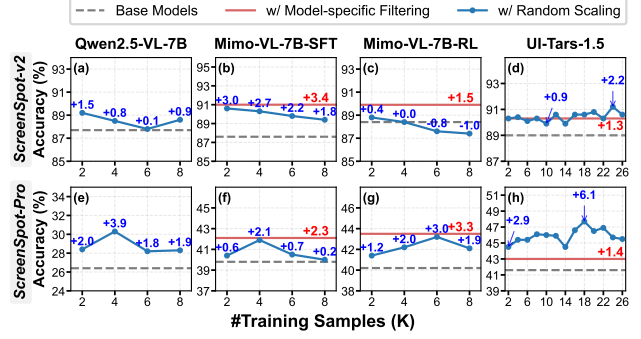


Figure 6. Effects of random data scaling and model-specific training data filtering on GUI grounding performance across models.

As shown in Table 4, training with unfiltered MobileViews data already improves all models over their baselines, indicating the inherent quality of the MobileViews dataset. Applying model-specific filtering yields further gains: on ScreenSpot-v2, improvements range from 1.3% to 3.4%, while on ScreenSpot-Pro, gains span 1.4% to 3.3%. For general-purpose models, MiMo-VL-7B-SFT and MiMo-VL-7B-RL, filtering removes noisy samples and stabilizes optimization, leading to consistent performance increases across both benchmarks. For the GUI-specific model UI-Tars-1.5, which is trained with only 254 samples, filtering still yields over a 1% improvement on both benchmarks, demonstrating its effectiveness under limited training data. These results confirm that MobileViews inherently provides effective data for training, while filtering further amplifies these benefits, enabling more efficient and robust model improvement.

**Impact of training data filtering strategy and data scale.** Figure 6 compares the results of random data scaling and model-specific data filtering across four VLMs. For general-purpose models, performance under random scaling tends to fluctuate or even decline as the training data increases, suggesting that simply enlarging the dataset introduces redundancy and noise. In contrast, model-specific filtering yields more stable improvements of 0.9–3.3% on both benchmarks for all models, showing that aligning data selection with each model’s learning capacity provides more effective supervision. On ScreenSpot-v2, UI-Tars-1.5 trained with only 254 filtered samples performs comparably to that trained with 26K random samples, confirming that targeted filtering can match or exceed the benefits of large-scale random data. These results demonstrate that model-specific training data filtering is more effective than simply scaling training data with random samples. Such filtering is enabled by the large scale and diversity of MobileViews, which provides a rich pool for extracting model-specific samples tailored to different VLMs’ learning capacities.



## 6. Conclusion

We develop MobileViews Crawler, a scalable mobile app traversal framework for large-scale data collection. Building on this, we construct MobileViews, an open-source dataset of 1.2M unique screenshot-VH pairs from over 30K modern apps with comprehensive metadata and bilingual support. Experiments show that MobileViews consistently improves both general-purpose and GUI-specific VLMs. Model-aware training data filtering further enhancing training efficiency. We believe that MobileViews can serve as a reliable foundation for advancing mobile GUI agents.

## References

- [1] Ui/application exerciser monkey. <https://developer.android.com/studio/test/other-testing-tools/monkey>, 2023. 2
- [2] Android debug bridge (adb). <https://developer.android.com/tools/adb>, 2024. 4
- [3] Qualcomm snapdragon processors. <https://www.qualcomm.com/snapdragon/overview>, 2025. 2, 5
- [4] Gilles Baechler, Srinivas Sunkara, Maria Wang, Fedir Zubach, Hassan Mansoor, Vincent Etter, Victor Cărbune, Jason Lin, Jindong Chen, and Abhanshu Sharma. Screenai: A vision-language model for ui and infographics understanding, 2024. 2
- [5] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025. 1, 2, 7, 8
- [6] Johannes Buchner. Imagehash: An image hashing library written in python. <https://github.com/JohannesBuchner/imagehash>, 2024. [Online; accessed September 1, 2024]. 2, 6
- [7] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, et al. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 24185–24198, 2024. 1
- [8] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9313–9332, Bangkok, Thailand, 2024. Association for Computational Linguistics. 7, 8
- [9] Biplab Deka, Zifeng Huang, Chad Franzen, Joshua Hibschman, Daniel Afergan, Yang Li, Jeffrey Nichols, and Ranjitha Kumar. Rico: A mobile app dataset for building data-driven design applications. In *Proceedings of the 30th annual ACM symposium on user interface software and technology*, pages 845–854, 2017. 1, 2, 3, 6
- [10] Longxi Gao, Li Zhang, and Mengwei Xu. Uishift: Enhancing vlm-based gui agents through self-supervised reinforcement learning. *arXiv preprint arXiv:2505.12493*, 2025. 6, 7
- [11] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for GUI agents. In *The Thirteenth International Conference on Learning Representations*, 2025. 8
- [12] Zhangxuan Gu, Zhengwen Zeng, Zhenyu Xu, Xingran Zhou, Shuheng Shen, Yunfei Liu, Beitong Zhou, Changhua Meng, Tianyu Xia, Weizhi Chen, et al. Ui-venus technical report: Building high-performance ui agents with rft. *arXiv preprint arXiv:2508.10833*, 2025. 3, 6, 7, 8
- [13] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. 6, 7
- [14] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14281–14290, 2024. 8
- [15] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14281–14290, 2024. 1
- [16] Yu-Chung Hsiao, Fedir Zubach, Maria Wang, et al. Screenqa: Large-scale question-answer pairs over mobile app screenshots. *arXiv preprint arXiv:2209.08199*, 2022. 3
- [17] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use. *arXiv preprint arXiv:2504.07981*, 2025. 2, 7
- [18] Wei Li, William E. Bishop, Alice Li, Christopher Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on UI control agents. In *NeurIPS*, 2024. 1, 2, 3
- [19] Yuanchun Li, Ziyue Yang, Yao Guo, and Xiangqun Chen. Droidbot: a lightweight ui-guided test input generator for android. In *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, pages 23–26. IEEE, 2017. 2, 4
- [20] Yang Li, Jiacong He, Xin Zhou, Yuan Zhang, and Jason Baldridge. Mapping natural language instructions to mobile ui action sequences. *arXiv preprint arXiv:2005.03776*, 2020. 2, 3
- [21] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 19498–19508, 2025. 1, 8
- [22] Yuhang Liu, Pengxiang Li, Congkai Xie, Xavier Hu, Xiaotian Han, Shengyu Zhang, Hongxia Yang, and Fei Wu. Infogui-r1: Advancing multimodal gui agents from reactive actors

- to deliberative reasoners. *arXiv preprint arXiv:2504.14239*, 2025. 6, 7, 8
- [23] Quanfeng Lu, Wenqi Shao, Zitao Liu, Lingxiao Du, Fanqing Meng, Boxuan Li, Botong Chen, Siyuan Huang, Kaipeng Zhang, and Ping Luo. Guidyssey: A comprehensive dataset for cross-app gui navigation on mobile devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 22404–22414, 2025. 1, 2, 3
- [24] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Guanqing Xiong, and Hongsheng Li. Ui-r1: Enhancing action prediction of gui agents by reinforcement learning. *arXiv preprint arXiv:2503.21620*, 2025. 6, 7, 8
- [25] OpenAI. Hello gpt-4o. <https://openai.com/index/hello-gpt-4o/>, 2024. 4, 6
- [26] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025. 2, 7, 8
- [27] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Androidinthewild: A large-scale dataset for android device control. *Advances in Neural Information Processing Systems*, 36:59708–59728, 2023. 1, 2, 3, 6
- [28] Haozhan Shen, Peng Liu, Jingcheng Li, Chunxin Fang, Yibo Ma, Jiajia Liao, Qiaoli Shen, Zilun Zhang, Kangjia Zhao, Qianqian Zhang, Ruochen Xu, and Tiancheng Zhao. Vlm-r1: A stable and generalizable r1-style large vision-language model. *arXiv preprint arXiv:2504.07615*, 2025. 7
- [29] Liangtai Sun, Xingyu Chen, Lu Chen, Tianle Dai, Zichen Zhu, and Kai Yu. Meta-gui: Towards multi-modal conversational agents on mobile gui. *arXiv preprint arXiv:2205.11029*, 2022. 2
- [30] Daniel Toyama, Philippe Hamel, Anita Gergely, Gheorghe Comanici, Amelia Glaese, Zafarali Ahmed, Tyler Jackson, Shibl Mourad, and Doina Precup. Androidenv: A reinforcement learning platform for android. *arXiv preprint arXiv:2105.13231*, 2021. 2
- [31] Bryan Wang, Gang Li, Xin Zhou, Zhourong Chen, Tovi Grossman, and Yang Li. Screen2words: Automatic mobile ui summarization with multimodal learning. In *The 34th Annual ACM Symposium on User Interface Software and Technology*, pages 498–510, 2021. 2, 3
- [32] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. Autodroid: Llm-powered task automation in android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*, pages 543–557, 2024. 2
- [33] Qinzhuo Wu, Weikai Xu, Wei Liu, Tao Tan, Jianfeng Liu, Ang Li, Jian Luan, Bin Wang, and Shuo Shang. Mobilevlm: A vision-language model for better intra- and inter-ui understanding. In *EMNLP (Findings)*, pages 10231–10251. Association for Computational Linguistics, 2024. 3
- [34] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. Os-atlas: A foundation action model for generalist gui agents. *arXiv preprint arXiv:2410.23218*, 2024. 2, 7, 8
- [35] Xiaobo Xia and Run Luo. Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458*, 2025. 6, 7
- [36] LLM-Core-Team Xiaomi. Mimo-vl technical report. *arXiv preprint arXiv:2506.03569*, 2025. 1, 2, 7, 8
- [37] Bofei Zhang, Zirui Shang, Zhi Gao, Wang Zhang, Rui Xie, Xiaojian Ma, Tao Yuan, Xinxiao Wu, Song-Chun Zhu, and Qing Li. Tongui: Building generalized gui agents by learning from multimodal web tutorials. *arXiv preprint arXiv:2504.12679*, 2025. 2, 8
- [38] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. Appagent: Multimodal agents as smartphone users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–20, 2025. 1
- [39] Li Zhang, Zhe Fu, Boqing Shi, Xiang Li, Rujin Lai, Chenyang Yang, Ao Zhou, Xiao Ma, Shangguang Wang, and Mengwei Xu. More is different: Prototyping and analyzing a new form of edge server with massive mobile socs. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 285–302, 2024. 2, 5
- [40] Li Zhang, Shihe Wang, Xianqing Jia, Zhihan Zheng, Yunhe Yan, Longxi Gao, Yuanchun Li, and Mengwei Xu. Llam-atouch: A faithful and scalable testbed for mobile ui task automation. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, pages 1–13, 2024. 2, 3